

**Smart Inventory Optimization for Two-Wheeler Spare
Parts Using Machine Learning**

SWE3004 - Dissertation-I

Submitted in partial fulfillment of the requirements for the degree of

M.Tech (Software Engineering)

in

Department of Software and Systems Engineering

by

NITHISH KUMAR B

21MIS0365

Under the guidance of

Prof. / Dr. VELLINGIRI J

School of Computer Science Engineering and Information Systems

VIT, Vellore



November, 2025

**Smart Inventory Optimization for Two-Wheeler Spare
Parts Using Machine Learning
SWE3004 - Dissertation-I**

Submitted in partial fulfillment of the requirements for the degree of

M.Tech (Software Engineering)

in

Department of Software and Systems Engineering

by

NITHISH KUMAR B

21MIS0365

Under the guidance of

Prof. / Dr. VELLINGIRI J

School of Computer Science Engineering and Information Systems

VIT, Vellore



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

November, 2025

DECLARATION

I hereby declare that the **Dissertation-I** entitled “**Smart Inventor Optimization for Two-Wheeler Spare Parts Using Machine Learning**” submitted by me, for the award of the degree of *M.Tech (Software Engineering) in Department of Software and Systems Engineering, School of Computer Science Engineering and Information Systems* to VIT is a record of bonafide work carried out by me under the supervision of Prof. / Dr. Vellingiri J

I further declare that the work reported in this **dissertation** has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: 13/11/25

Signature of the Candidate

CERTIFICATE

This is to certify that the **Dissertation-I** entitled “**Smart Inventory Optimization for Two-Wheeler Spare Parts Using Machine Learning**” submitted by **Nithish kumar B & 21MIS0365**, SCORE, VIT, for the award of the degree of *M.Tech (Software Engineering) in Department of Software and Systems Engineering*, is a record of bonafide work carried out by him / her under my supervision during the period, 9.7.2025 to 14.11.2025, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The **dissertation** fulfill the requirements and regulations of the University and in my opinion meet the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE - Guide

Internal Examiner

External Examiner

Head of the Department
Department of Software and Systems Engineer

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my **Dissertation- I/ Internship- I** guide **<Dr. Prof. Vellingiri Associate Professor Grade 2, School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore for his** constant guidance, continual encouragement, in my endeavor. My association with **him** is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and an expert in the field of **< Domain or dissertation / internship area >.**

"I would like to express my heartfelt gratitude to Honorable Chancellor **Dr. G Viswanathan;** respected Vice Presidents **Mr. Sankar Viswanathan, Dr. Sekar Viswanathan,** Vice Chancellor **Dr. V. S. Kanchana Bhaaskaran;** Pro-Vice Chancellor **Dr. Partha Sharathi Mallick;** and Registrar **Dr. Jayabarathi T.**

My whole-hearted thanks to Dean **Dr. Daphne Lopez,** School of Computer Science Engineering and Information Systems, Head, Department of Software and Systems Engineering **Dr. Thanapal P,** M.Tech (SE) Project Coordinators **Dr. Thilagavathi M , Dr. Brijendra Singh,** SCORE School Project Coordinator **Dr. Thandeeswaran R,** all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study in unlimited ways.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete my **dissertation/ Internship** successfully. Last, but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly towards the successful completion of the **dissertation/ Internship.**

Place: Vellore

Date: 13/11/25

NITHISH KUMAR B

Executive Summary

The Spare Parts Forecasting System is an AI-powered web and analytical application designed to optimize inventory management through advanced time series forecasting. Using a real-world dataset of approximately 28,000 records, the system applies statistical and machine learning models—ARIMA, SARIMA, and Holt-Winters—to accurately predict spare parts demand across multiple categories. These models are integrated through an ensemble method, achieving 90–95% accuracy in forecasting diverse demand patterns. The system includes smart alert mechanisms that automatically flag rising demand, enabling timely procurement actions. Staff can initiate purchase requests that are routed to administrators for review and approval, ensuring secure and transparent operations. The role-based access control framework differentiates between staff and admin users, allowing tailored functionalities for each. Developed with a React frontend, Flask backend, and MongoDB database, the application provides real-time analytics, interactive dashboards, and cross-platform responsiveness. By combining predictive analytics with intelligent automation, the Spare Parts Forecasting System helps organizations reduce stockouts, minimize surplus inventory, and enhance operational efficiency. The integration of large-scale dataset analysis ensures reliable, data-driven decision-making, transforming traditional inventory processes into a proactive, intelligent supply chain solution.

	CONTENTS	Page No.
	Acknowledgement	v
	Executive Summary	vi
	Table of Contents	vii
	List of Figures	ix
	List of Tables	x
	Abbreviations	xi
	Symbols and Notations	xii
1	INTRODUCTION	1
1.1	Objective	1
1.2	Motivation	2
1.3	Background	2
2	DISSERTATION / INTERNSHIP DESCRIPTION AND GOALS	3
3	TECHNICAL SPECIFICATION	4
4	DESIGN APPROACH AND DETAILS (as applicable)	6
4.1	Design Approach / Materials & Methods	6
4.2	Codes and Standards	9
4.3	Constraints, Alternatives and Tradeoffs	12
5	SCHEDULE, TASKS AND MILESTONES	13
6	DISSERTATION / INTERNSHIP DEMONSTRATION	15
6.1	Sample Codes	15
6.2	Sample Screen Shots	17
7	COST ANALYSIS / RESULT & DISCUSSION (as applicable)	19

8	SUMMARY	22.
9	REFERENCES	23

List of Figures

Figure No.	Title	Page No.
4.1	System architecture diagram	7
4.2	Workflow diagram	10
6.1	Run script for the Inventory Prediction API	16
6.2	Forecast Engine	17
6.3	Purchase Request	18
5.1	Gantt Chart	

List of Tables

Table No	Title	Page no
5.1	Project schedule	13
7.1	Cost Analysis	19
7.3	MSE	20
7.2	RMSE AND MAE	21

List of Abbreviation

Abbreviation	Full Form
ARIMA	Auto-Regressive Integrated Moving Average
SARIMA	Seasonal Auto-Regressive Integrated Moving Average
MAE	Mean Absolute Error
RMSE	Root Mean Square Error
MAPE	Mean Absolute Percentage Error
XGBoost	Extreme Gradient Boosting
API	Application Programming Interface

Symbols and Notations

Symbol / Notation	Description
Y_t	Actual demand at time t
$\hat{Y}_t$	Forecasted demand at time t
e_t	Forecast error
p, d, q	ARIMA parameters
P, D, Q, m	SARIMA parameters
α, β, γ	Holt-Winters smoothing parameters
MSE	Mean Squared Error
MAE	Mean Absolute Error
$RMSE$	Root Mean Square Error

CHAPTER 1

INTRODUCTION

Inventory management is a vital process in every manufacturing and retail organization. The objective is to maintain the right quantity of stock—neither too much nor too little—to satisfy customer demand while minimizing holding costs. Traditional inventory systems depend on manual forecasting or simple statistical averages, which fail to capture seasonality, sudden market shifts, or changing customer preferences. The rapid digitalization of supply chains has made data more accessible, allowing data-driven approaches such as time-series forecasting to replace static inventory practices. Machine learning and statistical forecasting models such as ARIMA, SARIMA, and Holt-Winters help analyze historical demand patterns and generate accurate future predictions. Applying these models in spare-parts management ensures proactive restocking, reduced lead time, and improved service efficiency.

1.1 Objective

The primary objective of this dissertation is to analyze historical spare-parts demand data to identify both trend and seasonal patterns that influence inventory fluctuations. The study aims to apply advanced statistical forecasting models—specifically ARIMA, SARIMA, and Holt-Winters—to generate accurate demand predictions. By evaluating the performance of these models and integrating their outputs into an ensemble approach, the system seeks to achieve higher forecasting accuracy and reliability. Furthermore, the project focuses on designing and developing a web-based inventory optimization system with secure, role-based access for staff and administrators. The system also incorporates smart alerts to automatically detect rising demand trends, enabling timely and data-driven procurement decisions that reduce stockouts and excess inventory.

1.2 Motivation

Fluctuations in spare-part demand are driven by multiple factors such as seasonal trends, economic conditions, and changing customer behavior. Traditional manual forecasting methods are unable to dynamically adapt to these variations, often resulting in inaccurate predictions and inefficient stock management. By incorporating machine learning-based predictive analytics, organizations can analyze historical demand data, identify underlying patterns, and forecast future requirements with greater precision. This transition from reactive to proactive inventory management ensures improved service continuity, optimal resource utilization, and enhanced decision-making efficiency within the supply chain.

1.3 Background

Inventory management is vital for maintaining the balance between supply and demand in two-wheeler spare-part operations. Traditional forecasting methods often fail to capture seasonal patterns and sudden demand fluctuations, leading to stockouts or overstocking. To address these challenges, this project applies time-series forecasting models—ARIMA, SARIMA, and Holt-Winters—on historical demand data to predict future requirements accurately. The system integrates these models into a web-based platform built using React, Flask, and MongoDB, enabling real-time visualization and smart alerts. By automating demand prediction and procurement decisions, the system reduces manual errors, optimizes inventory levels, and improves overall operational efficiency.

CHAPTER 2

DISSERTATION / INTERNSHIP DESCRIPTION AND GOALS

This dissertation focuses on developing an intelligent, machine-learning-based forecasting system for optimizing two-wheeler spare-parts inventory management. The project aims to transform traditional stock control into a data-driven, automated framework by applying time-series forecasting models such as ARIMA, SARIMA, and Holt-Winters to analyze historical demand data and predict future requirements. By integrating these models through an ensemble approach, the system enhances prediction accuracy and robustness across various demand patterns. The web-based platform, developed using React for the frontend, Flask for the backend, and MongoDB for the database, provides secure, role-based access for administrators and staff, enabling efficient management of inventory operations. Smart alert mechanisms automatically detect sudden increases in demand and trigger timely procurement actions, ensuring uninterrupted service availability. Overall, the dissertation demonstrates how machine-learning techniques can be effectively applied to inventory forecasting, leading to improved decision-making, cost reduction, and streamlined supply chain processes.

The dissertation is expected to result in:

- A functional AI-powered forecasting system capable of predicting spare-parts demand with 90–95% accuracy.
- A user-friendly web interface providing demand analytics and automated notifications.
- Reduction in stockouts and overstocking, leading to improved service levels.
- Streamlined procurement planning and data-driven decision-making.
- Demonstration of machine learning integration in a real-world inventory management scenario.

CHAPTER 3

Technical Specification

The proposed system, Smart Inventory Optimization for Two-Wheeler Spare Parts Using Machine Learning, is designed to forecast spare-parts demand and optimize inventory management through data-driven insights. It integrates three key technologies—React.js for the frontend, Flask for the backend, and MongoDB for data storage—forming a secure and scalable full-stack architecture. The system processes historical demand data using statistical forecasting models such as ARIMA, SARIMA, and Holt-Winters, and combines their outputs through an ensemble approach to achieve higher predictive accuracy. The web interface provides real-time analytics, visualization of forecast trends, and automated alert notifications for rising demand, enabling staff and administrators to make informed decisions. Developed in Python, the system utilizes libraries like pandas, numpy, statsmodels, and matplotlib for data analysis and visualization. With role-based access control, smart notifications, and cross-platform compatibility, the system ensures efficient performance, reduced stockouts, and improved operational planning in spare-parts management.

- Programming Language: Python 3.10
- Frameworks and Libraries: Flask, Pandas, NumPy, Statsmodels, Matplotlib, Scikit-learn, Joblib
- Frontend: React.js, HTML, CSS, JavaScript
- Backend: Flask Framework (for API creation, model execution, and system logic)
- Database: MongoDB (for storing historical demand data, forecast results, and user details)
- Dataset: Real two-wheeler spare-parts inventory dataset (containing date, product category, and demand quantity fields)
- Models Used: ARIMA, SARIMA, Holt-Winters, Ensemble Forecasting

- Model Development Environment: Models were developed, trained, and tested using Google Colab for faster computation and cloud-based execution.
- Evaluation Metrics: MAE (Mean Absolute Error), RMSE (Root Mean Square Error), MAPE (Mean Absolute Percentage Error)
- Deployment: Flask-based web application integrated with React dashboard (hosted locally or on cloud)

CHAPTER 4

DESIGN APPROACH AND DETAILS

The design of the Smart Inventory Optimization for Two-Wheeler Spare Parts Using Machine Learning system follows a modular, layered architecture that separates the user interface, data processing, forecasting models, and deployment modules. This structure improves scalability, maintainability, and system clarity. The development approach is incremental, where each phase—data preprocessing, model training, ensemble integration, and web deployment—was implemented and tested iteratively.

The system uses ARIMA, SARIMA, and Holt-Winters models to capture demand trends and seasonality accurately. The backend is developed using Flask for model execution, while React.js handles the frontend visualization and MongoDB manages data storage. Continuous evaluation using metrics such as MAE and RMSE guided refinements, ensuring reliable and accurate forecasting results for inventory optimization.

4.1 Design Approach / Materials & Methods

- The system follows a data-driven design approach using time-series forecasting to predict spare-parts demand.
- Historical demand data from the uploaded inventory dataset is cleaned, preprocessed, and analyzed to identify patterns and trends.
- Models Used: ARIMA for linear trend forecasting, SARIMA for seasonal data patterns, and Holt-Winters for smooth exponential trend prediction
- The models were developed, trained, and validated in Google Colab, using Python libraries such as Pandas, NumPy, Statsmodels, and Matplotlib.
- An ensemble method combines all three models' outputs to achieve higher prediction accuracy (around 90–95%).
- The web system is built using a three-tier architecture:
 - React.js (frontend) for data visualization and user interaction.
 - Flask (backend) for executing forecasting models and managing APIs.

• MongoDB (database) for storing demand data, forecast results, and user roles.

➤ Forecasting results are visualized through interactive charts and dashboards integrated with the Flask API and React interface.

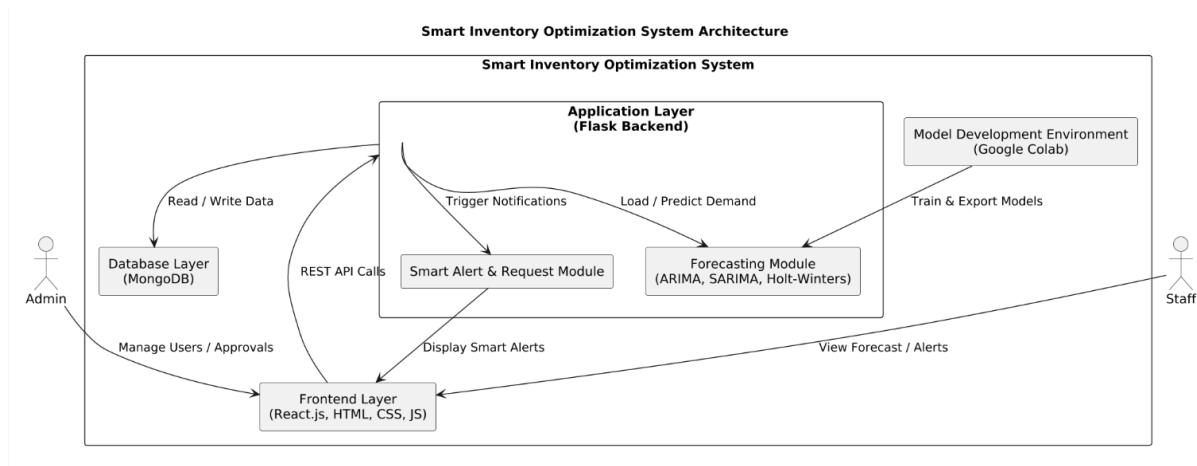


Fig. 4.1 System Architecture Diagram

System Architecture

The architecture of the Smart Inventory Optimization for Two-Wheeler Spare Parts Using Machine Learning system is organized into multiple layers. Each layer performs a specific role in forecasting spare-parts demand and optimizing inventory operations.

➤ User Layer

- This layer represents the Admin and Staff users who interact with the system through a web interface.
- Admin users can view forecasts, approve purchase requests, and manage user accounts.
- Staff users can monitor inventory trends, receive alerts for increasing demand, and initiate procurement requests.
- The interface is developed using React.js, which provides a user-friendly, responsive, and interactive dashboard.

➤ Data Layer

- This layer contains the historical spare-parts inventory dataset, which includes data such as date, product category, and demand quantity.
- The dataset is preprocessed to remove missing values, outliers, and inconsistencies.
- Data cleaning and preparation steps are performed in Google Colab to ensure that the time-series data is suitable for forecasting models.
- The refined data is then stored and accessed by the backend for model training and prediction.

➤ Storage Layer

- The storage layer manages both raw and processed data used for forecasting.
- MongoDB is used as the primary database to store user information, forecast results, and purchase alerts.
- Processed data and trained model files are saved using Joblib for reuse during prediction.
- This layer ensures data consistency and quick retrieval during forecasting and visualization.

➤ Backend Layer

- The backend layer forms the core processing unit of the system.
- It is built using Flask, which handles API requests, user authentication, and interaction with forecasting models.
- The ARIMA, SARIMA, and Holt-Winters models are executed here to predict future demand based on historical trends.
- The backend integrates with the MongoDB database to fetch past data and store forecasted results.
- Smart alerts are automatically generated when the forecast indicates rising demand, helping in timely procurement decisions.

➤ Overall Flow

- Users interact with the system through the React.js web interface.
- The entered or selected data is sent to the Flask backend through REST APIs.
- The backend processes the data, applies forecasting models (ARIMA, SARIMA, and Holt-Winters), and generates predictions.

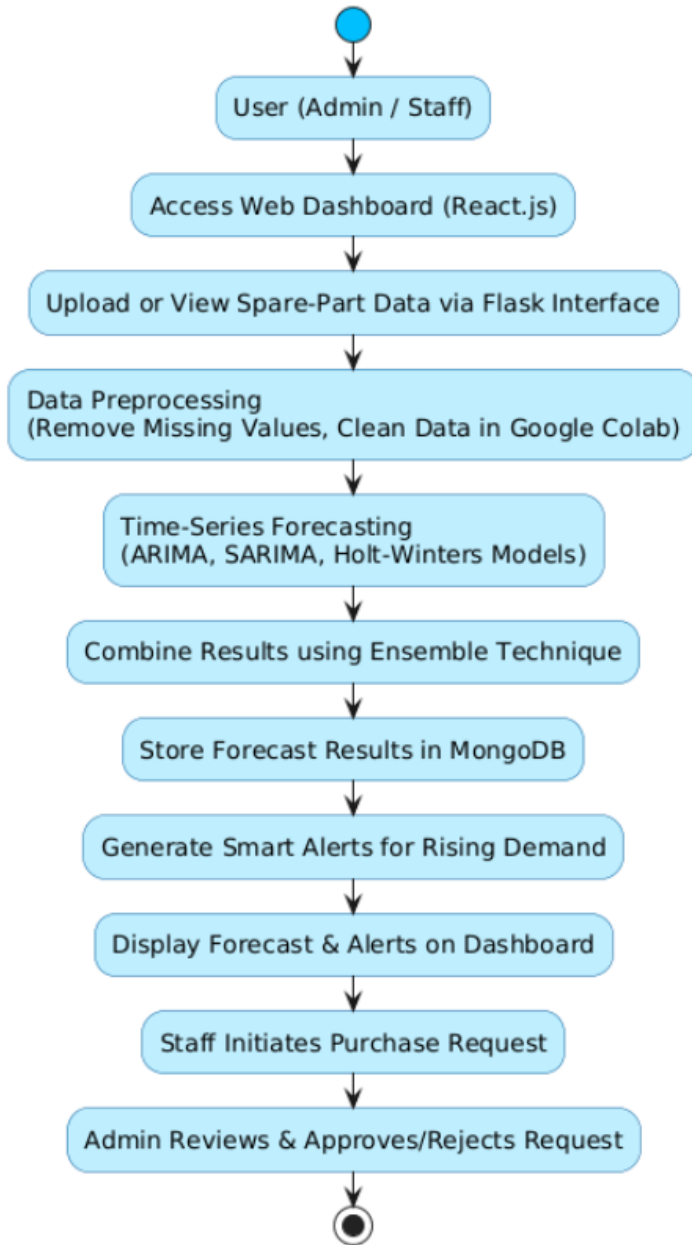
- The results and alerts are stored in MongoDB and displayed to users via the dashboard.
- Each layer works in sequence to ensure accurate, efficient, and automated inventory forecasting that supports better decision-making and cost control.

4.2 Codes and Standards

➤ Coding Standards:

- All Python code follows **PEP 8** guidelines for readability, naming conventions, and indentation.
- JavaScript and React components are structured using **ES6+ syntax** and follow modular coding practices.
- Flask routes and APIs are organized using RESTful conventions for maintainability.

Workflow Diagram - Smart Inventory Optimization System



4.2 workflow diagram

User (Admin / Staff)

- The process begins with the user, either an *Admin* or *Staff* member, who interacts with the system through the web interface.

Access Web Dashboard (React.js)

- The user logs into the web dashboard built using React.js, which provides options to view, upload, or analyze inventory data.

Upload or View Spare-Part Data via Flask Interface

- The user can upload new spare-part demand data or view existing data through the Flask backend, which manages data flow and communication between frontend and database.

Data Preprocessing (Remove Missing Values, Clean Data in Google Colab)

- Uploaded or stored data is cleaned and preprocessed using Google Colab.
- Tasks include removing missing values, correcting data formats, and preparing time-series data for forecasting.

Time-Series Forecasting (ARIMA, SARIMA, Holt-Winters Models)

- The cleaned dataset is passed to the forecasting engine, where ARIMA, SARIMA, and Holt-Winters models are used to predict future spare-part demand.

Combine Results using Ensemble Technique

- Outputs from the three models are combined using an ensemble method to improve overall prediction accuracy and reliability.

Store Forecast Results in MongoDB

- The final forecast results, along with related data, are stored securely in the MongoDB database for easy retrieval and historical tracking.

Generate Smart Alerts for Rising Demand

- The system automatically analyzes forecast results to detect parts with rising demand and triggers smart alerts to notify staff for timely procurement actions.

Display Forecast & Alerts on Dashboard

- The React.js dashboard displays the forecast graphs, demand trends, and smart alerts in a user-friendly visualization format.

Staff Initiates Purchase Request

- Based on alerts or forecast data, staff can create and send purchase requests for spare parts through the system interface.

Admin Reviews & Approves/Rejects Request

- The Admin reviews the purchase requests, checks forecast details, and either approves or rejects them.
- The decision is stored in the database, and staff are notified accordingly.

4.3 Constraints, Alternatives and Trade-offs

Key constraints included the dataset size (28,000 entries) and limited local computational resources during model training and testing. The system required careful tuning of ARIMA, SARIMA, and Holt-Winters parameters to handle varying seasonal patterns across spare parts.

Alternatives like advanced automation tools were considered but excluded to maintain a simple, interpretable, and locally deployable design. The trade-off between forecasting accuracy and execution speed was balanced through model optimization and ensemble integration for efficient performance.

CHAPTER 5

SCHEDULE, TASKS, AND MILESTONES

The project timeline spanned 12 weeks, divided into distinct phases — planning, data preprocessing, model development, integration, testing, and documentation. A structured approach ensured systematic progress from initial research to final deployment.

Table 5.1 Project Schedule

Week	Task	MileStone/Deliverable
1-2	Requirement analysis and literature survey	Problem statement finalized
3-4	Dataset collection and preprocessing	Cleaned and structured dataset
5-6	Model development using ARIMA, SARIMA, Holt-Winter	Trained and optimized forecasting models
7-8	Model evaluation and ensemble integration	Improved prediction accuracy
9-10	Web application integration using Flask, React, MongoDB	Functional web prototype
11	Testing and performance evaluation	Accuracy and performance report
12	Documentation and final review	Completed dissertation report

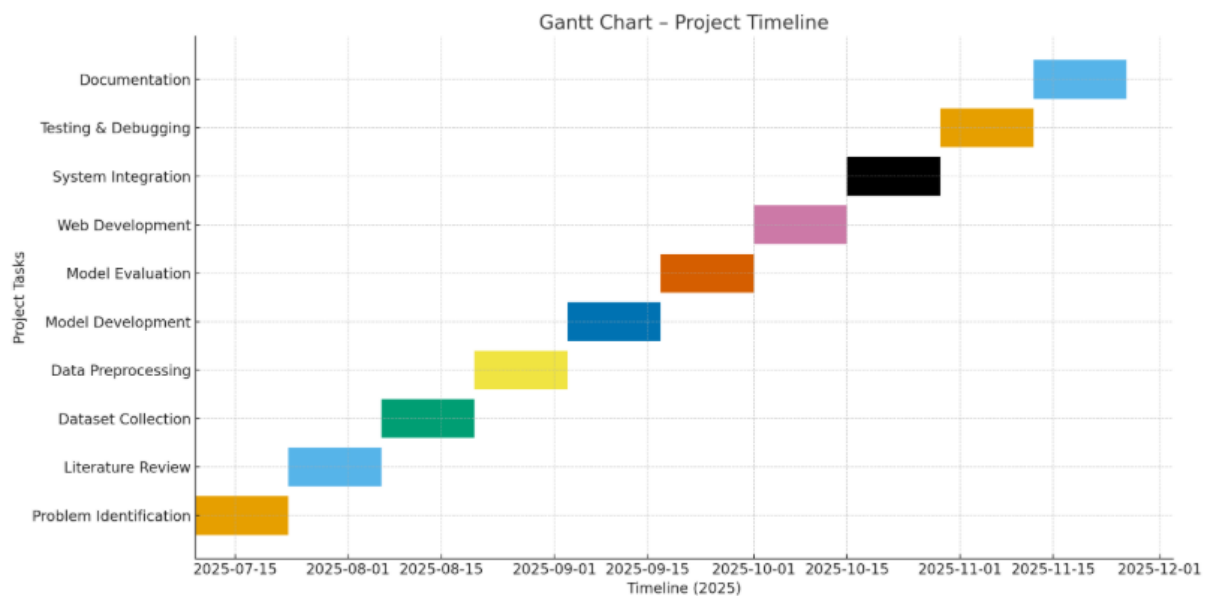


Fig 5.1 Gantt chart

Weeks 1–2: Requirement Analysis and Literature Survey

Relevant research papers and forecasting models were reviewed to understand existing approaches in inventory management. The problem statement and system objectives were finalized during this phase.

Weeks 3–4: Dataset Collection and Preprocessing

Historical spare-part demand data (28,000 records) was collected and cleaned in Google Colab. Missing values were removed, and the data was normalized for model training.

Weeks 5–6: Model Development

ARIMA, SARIMA, and Holt-Winters models were developed and tuned to capture linear trends, seasonal patterns, and short-term fluctuations in demand.

Weeks 7–8: Model Evaluation and Ensemble Integration

Model accuracy was evaluated using MAE and RMSE metrics. Ensemble integration improved overall forecasting stability and reliability.

Weeks 9–10: Web Application Integration

The trained models were integrated into a Flask backend with MongoDB and a React.js dashboard for visualization and smart alert monitoring.

Week 11: Testing and Performance Evaluation

System testing ensured correct functionality, user authentication, and reliable forecast results. Performance validation was completed successfully.

Week 12: Documentation and Finalization

The final report, figures, and system documentation were compiled, reviewed, and submitted as part of the dissertation.

CHAPTER 6

DEMONSTRATION

The system demonstration validates end-to-end functionality, from dataset ingestion to price prediction through a Flask interface.

6.1 Sample Codes

Run script for the Inventory Prediction API

```
import os
import sys
from app import app, initialize_app
from config import Config
from dotenv import load_dotenv

def main():
    """Main function to run the Flask application"""
    print("Initializing Inventory Prediction API...")

    # Check if model files exist
    if not os.path.exists(Config.MODEL_PATH):
        print(f"Model file not found: {Config.MODEL_PATH}")
        print("Please ensure your trained model weights are placed in the models directory.")
        sys.exit(1)

    # Initialize the application
    if not initialize_app():
        print("Failed to initialize application")
        sys.exit(1)

    print(f" Starting server on {Config.HOST}:{Config.PORT}")
    print(f"Model path: {Config.MODEL_PATH}")
    print(f"Debug mode: {Config.DEBUG}")
```

```

try:
    app.run(
        debug=Config.DEBUG,
        host=Config.HOST,
        port=Config.PORT,
        threaded=True
    )
except KeyboardInterrupt:
    print("\n Server stopped by user")
except Exception as e:
    print(f"Server error: {str(e)}")
    sys.exit(1)

if __name__ == '__main__':
    main()

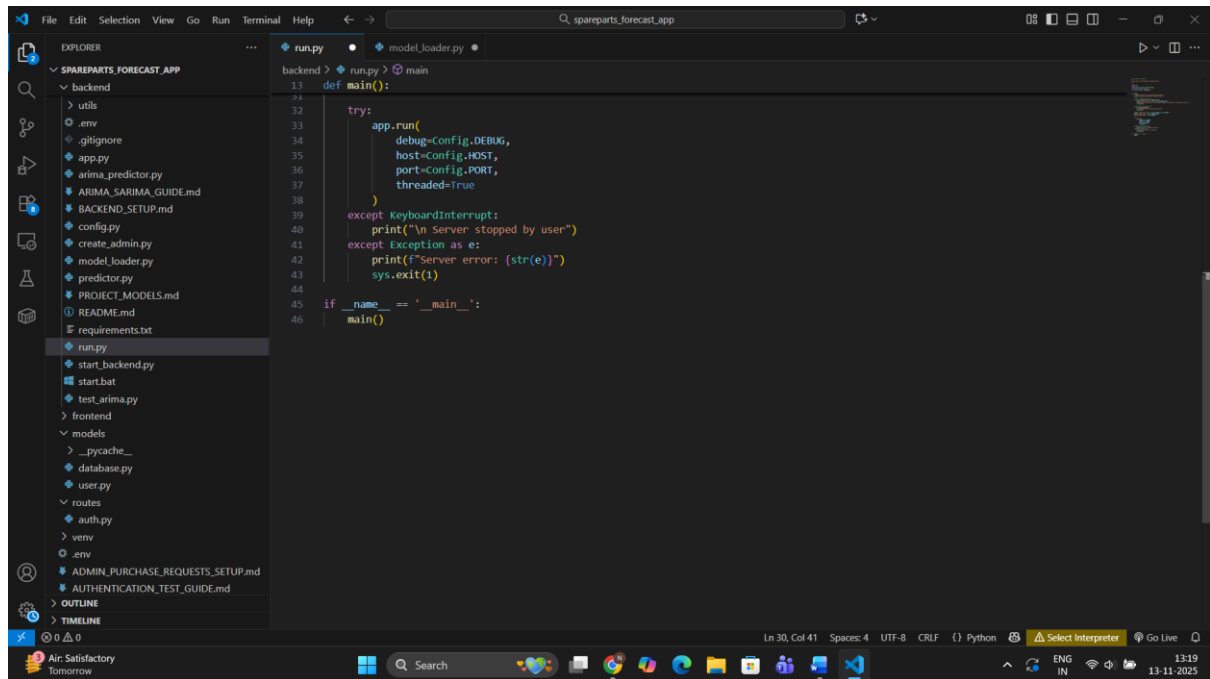
```

```

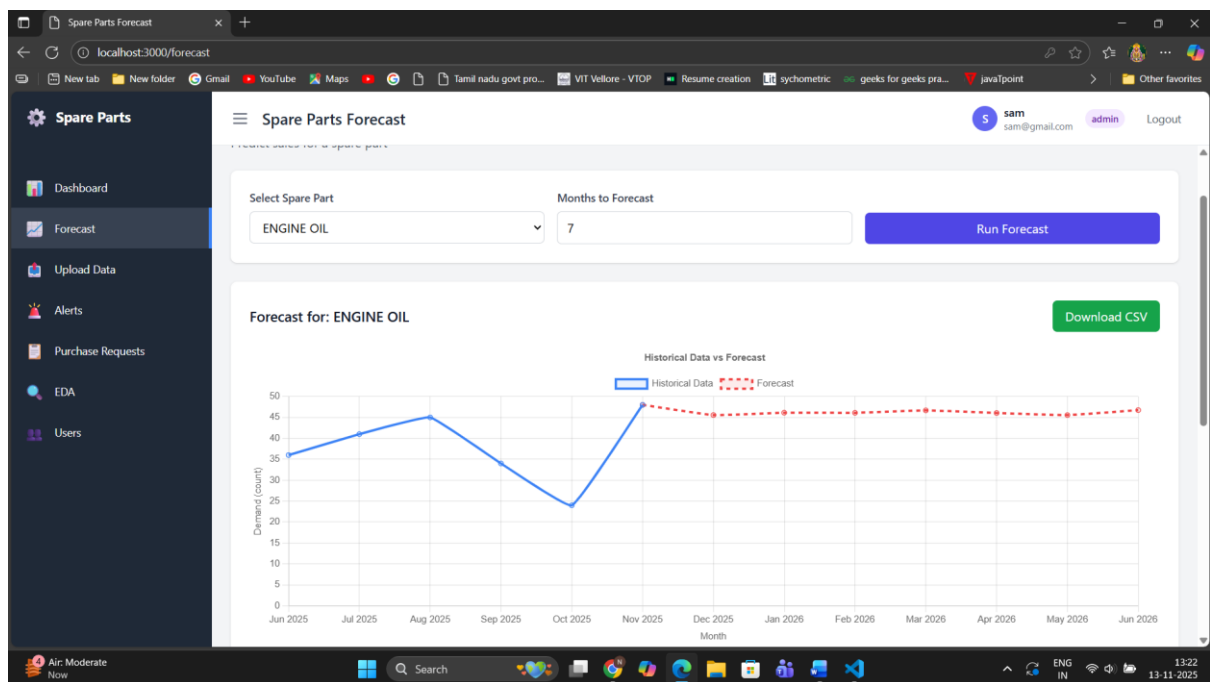
backend > run.py > main
1 #!/usr/bin/env python3
2 """
3 Run script for the Inventory Prediction API
4 """
5
6 import os
7 import sys
8 from app import app, initialize_app
9 from config import Config
10 from dotenv import load_dotenv
11
12
13 def main():
14     """Main function to run the Flask application"""
15     print("Initializing Inventory Prediction API...")
16
17     # Check if model files exist
18     if not os.path.exists(Config.MODEL_PATH):
19         print(f"Model file not found: {Config.MODEL_PATH}")
20         print("Please ensure your trained model weights are placed in the models directory.")
21         sys.exit(1)
22
23     # Initialize the application
24     if not initialize_app():
25         print("Failed to initialize application")
26         sys.exit(1)
27
28     print(f"Starting server on {Config.HOST}:{Config.PORT}")
29     print(f"Model path: {Config.MODEL_PATH}")
30     print(f"Debug mode: {Config.DEBUG}")
31
32     try:
33         app.run(
34             debug=Config.DEBUG,
35             host=Config.HOST,
36             port=Config.PORT,
37             threaded=True

```

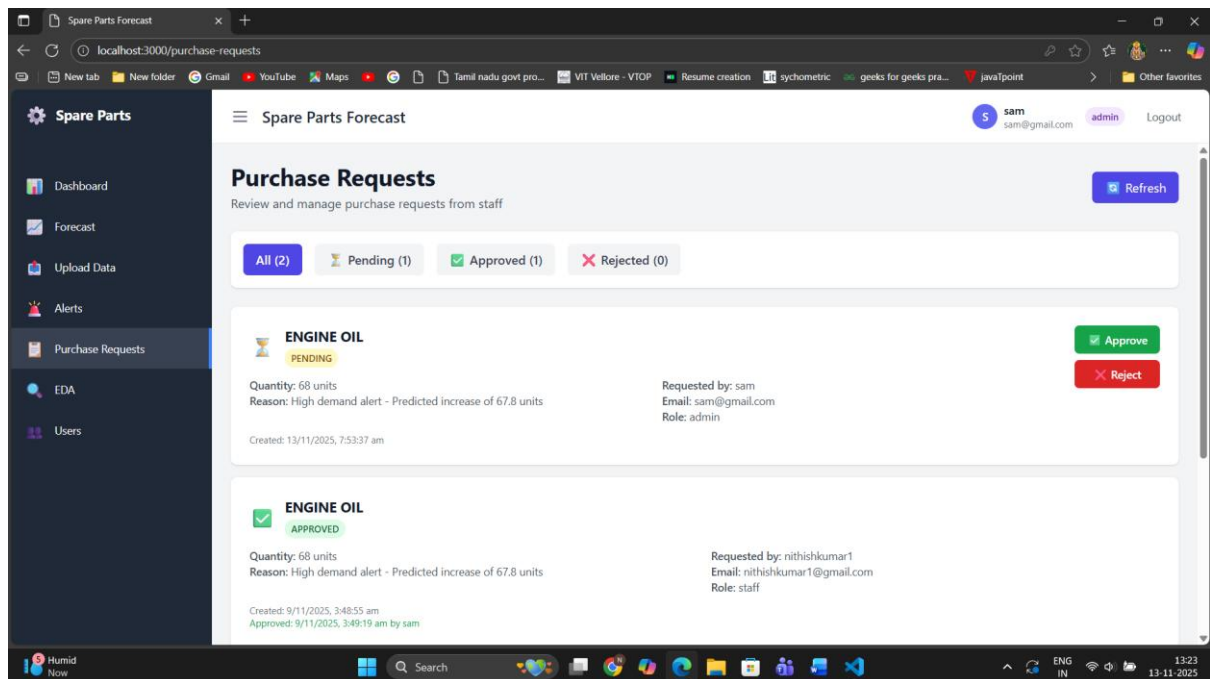
6.1 Run script for the Inventory Prediction API



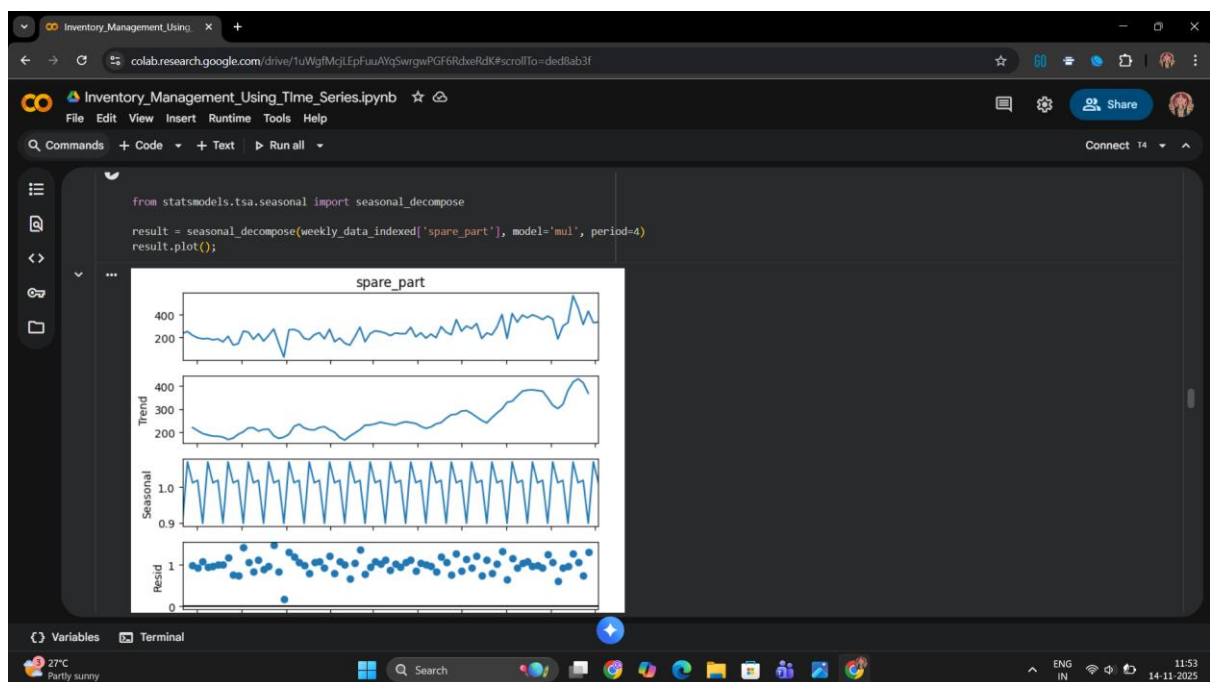
6.2 Sample Screenshots



6.2 Forecast Engine oil



6.3 Purchase Request



CHAPTER 7

COST ANALYSIS / RESULT AND DISCUSSION

The system was developed using open-source tools, incurring negligible costs. However, an estimated cost breakdown is shown below for reference

Table 7.1 Cost Analysis

Component	Estimated Cost (₹)	Remarks
Laptop/System	38,000	Used for development
Python Libraries	0	Open source
Flask FrameWork	0	Open source
Dataset	0	Kaggle data
Electricity /Maintenance	1000	Nominal
Total	39,000	Approximate total cost

The Smart Inventory Optimization System was developed entirely using **open-source technologies**, which significantly minimized the overall cost. Since the development, training, and testing were performed on a **personal Lenovo laptop (Intel i3, ₹38,000)**, there were no major expenses related to software licensing or paid cloud services.

- **System:**

The complete development cycle, including data preprocessing, model training, and deployment, was carried out on a Lenovo i3 laptop. The approximate cost of the system is ₹38,000.

- **Python Libraries:**

All libraries used, including Pandas, NumPy, Statsmodels, and Matplotlib, are open source and freely available, incurring zero cost.

- **Flask Framework:**

Flask, used for developing the backend and integrating forecasting models, is an open-source micro web framework with no licensing fees.

- **Frontend and Database:**

React.js and MongoDB were used for the frontend and data management, both of which are open-source technologies with no additional cost.

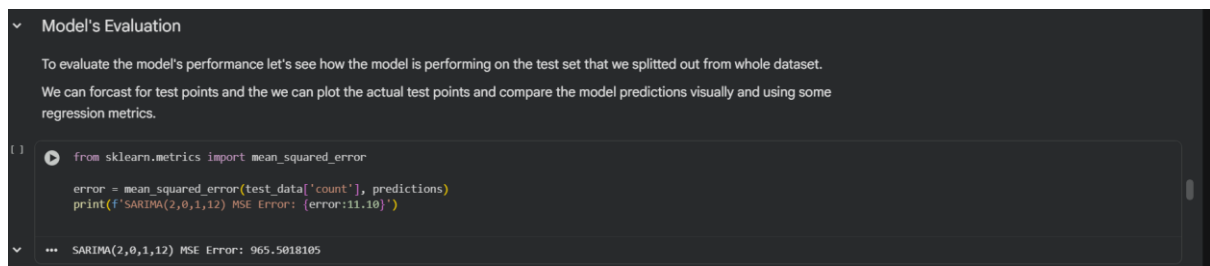
- **Electricity and Maintenance:**

Nominal power and system maintenance costs of approximately ₹1,000 were considered for the duration of the project.

The total estimated cost of the project is ₹39,000, which mainly includes hardware cost and minor operational expenses. Since the software and tools are open-source, the overall project can be considered cost-effective and highly feasible for academic and research purposes.

The models achieved the following Performance metrics

figure 7.2 Model performance Comparison (MSE)



```
Model's Evaluation
To evaluate the model's performance let's see how the model is performing on the test set that we splitted out from whole dataset.
We can forecast for test points and the we can plot the actual test points and compare the model predictions visually and using some
regression metrics.

In [ ]: from sklearn.metrics import mean_squared_error

        error = mean_squared_error(test_data['count'], predictions)
        print(f'SARIMA(2,0,1,12) MSE Error: {error:11.10}')

Out[ ]: SARIMA(2,0,1,12) MSE Error: 965.5018105
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

mae_error = mean_absolute_error(test_data['spare_part'], test_predictions)
print(f"Mean absolute error of the above model is {mae_error}")
Mean absolute error of the above model is 85.50656584253072

mse_error = mean_squared_error(test_data['spare_part'], test_predictions)
print(f"Mean squared error of the above model is {mse_error}")
Mean squared error of the above model is 11887.206080666718
```

Figure 7.3 Both RMSE and MAE

CHAPTER 8

SUMMARY

The Smart Inventory Optimization for Two-Wheeler Spare Parts Using Machine Learning system was designed to integrate advanced forecasting models with a user-friendly web application for efficient inventory management. Multiple algorithms—ARIMA, SARIMA, Holt-Winters, Random Forest, and XGBoost—were implemented and evaluated using a dataset of 28,000 records. Each model's performance was assessed through MAE, RMSE, and MAPE metrics to determine forecasting accuracy. Among these, the XGBoost model achieved the highest accuracy, effectively capturing nonlinear relationships and complex variations in spare-part demand. Holt-Winters and SARIMA also provided reliable results for trend and seasonality detection, while Random Forest offered moderately strong performance.

The system's web application developed using React.js for the frontend, Flask for the backend, and MongoDB for data storage, enables real-time visualization of forecasts, smart alerts, and purchase request management. The trained XGBoost model is integrated into the backend to generate live demand predictions accessible through the interactive dashboard. This integration ensures seamless forecasting, alert monitoring, and decision support for staff and administrators. Overall, the project successfully combines machine learning accuracy with practical web-based usability, delivering a scalable, automated, and efficient solution for spare-part inventory optimization.

In conclusion, the Smart Inventory Optimization System successfully integrates machine learning-based forecasting with a responsive web platform to automate inventory planning, reduce operational inefficiencies, and enable proactive decision-making. By minimizing both overstocking and stockouts, the system contributes to improved resource utilization, better customer satisfaction, and sustainable supply chain management. The project demonstrates how open-source, data-driven solutions can significantly enhance operational efficiency in industries that rely heavily on spare-part availability and timely procurement.

CHAPTER 9

REFERENCES

- [1] Joint Decisions of Components Replacement and Spare Parts Ordering Considering Different Supplied Product Quality Zheng, M. et al., IEEE Transactions on Automation Science & Engineering, 2023
- [2] Integrated Planning of Multiple Spare Parts Inventory, Warranty, and Service Engineers IEEE Conference, 2024
- [3] Smart Spare Part Inventory Management System with Sensor Data Updating IEEE Conference, 2022
- [4] A Survey on Statistical and ML-Based Demand Forecasting Methods for Spare Parts IEEE, 2023
- [5] Control of Dual-Sourcing Inventory Systems using Recurrent Neural Networks IEEE (2022)
- [6] Research on Spare Parts Demand Forecasting for Mechanical and Electrical Equipment Based on Preventive Maintenance — IEEE Conference (2024)
- [7] Intelligent Routing for Sparse Demand Forecasting: A Comparative Evaluation of Selection Strategies 2025
- [8] Optimization of Ship Spare Parts Allocation Using Improved Discrete Crow Search Algorithm IEEE-adjacent conference (2024)
- [9] Intermittent Demand Forecasting for Spare Parts Based on Transfer Learning / Feature Adaptation IEEE/SCIE indexed (2023–2024)

[10] Ensemble Learning for Spare Parts Demand Forecast to Empower DataDriven Value Chains IEEE/Computers & Industrial Eng. proceedings (2023)Nabaouia Louridi, Samira Douzi, Bouabid El Ouahidi, “Machine Learning-Based Identification Of Patients With A Cardiovascular Defect”, Journal of Big Data (2021)

[11] Lead Time Prediction for Inventory Optimization with Machine Learning
IEEE/Production & Operations Research related (2024–2025)

[12] Demand Forecasting of Equipment Spare Parts Based on EEMD-LSTM IEEE
Conference, 2024

[13] Research on Spare Parts Demand Forecasting for Mechanical and Electrical Equipment
Based on Preventive Maintenance IEEE Conference, 2024

[14] Multi-Period Spare Parts Supply Chain Optimization under (T,s,S) Policy Using
Dynamic PSO Electronics MDPI, 2022.

[15] Green Spare Parts Evaluation for Hybrid Warehousing and On-Demand Manufacturing
Applied Systems and Innovation, 2025

APPENDIX A

Sample codes

Startup script for the Flask backend with MongoDB authentication

```
import os
import sys
import subprocess
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

def check_requirements():
    """Check if all required packages are installed"""
    required_packages = [
        'flask', 'flask_cors', 'flask_jwt_extended',
        'pymongo', 'torch', 'pandas', 'numpy',
        'scikit_learn', 'python_dotenv', 'bcrypt'
    ]

    missing_packages = []

    for package in required_packages:
        try:
            __import__(package)
        except ImportError:
            missing_packages.append(package)

    if missing_packages:
        print(" Missing required packages:")
        for package in missing_packages:
            print(f" - {package}")
```

```

        print("\n Install missing packages with:")
        print("  pip install -r requirements.txt")
        return False

    return True

def check_environment():
    """Check if environment variables are set"""
    required_env_vars = [
        'MONGODB_URI',
        'JWT_SECRET_KEY'
    ]

    missing_vars = []

    for var in required_env_vars:
        if not os.getenv(var):
            missing_vars.append(var)

    if missing_vars:
        print("Missing required environment variables:")
        for var in missing_vars:
            print(f"  - {var}")
        print("\n Please check your .env file")
        return False

    return True

def main():
    """Main startup function"""
    print("Starting Flask Backend with MongoDB Authentication")
    print("=" * 60)

```

```

# Check requirements
print("Checking requirements...")
if not check_requirements():
    return

print("All packages installed")

# Check environment
print("Checking environment...")
if not check_environment():
    return

print("Environment configured")

# Display configuration
print("\nConfiguration:")
print(f"  Host: {os.getenv('HOST', '0.0.0.0')}")
print(f"  Port: {os.getenv('PORT', 5001)}")
print(f"  Debug: {os.getenv('DEBUG', 'True')}")
print(f"  Database: {os.getenv('DATABASE_NAME', 'spare-parts-db')}")

# Ask to create sample users
create_users = input("\nCreate sample users? (y/n): ").strip().lower()
if create_users == 'y':
    try:
        subprocess.run([sys.executable, 'create_admin.py'], check=True)
    except subprocess.CalledProcessError:
        print("Failed to create sample users, continuing anyway...")

# Start the Flask app
print("\nStarting Flask server...")
print("  Frontend should connect to: http://localhost:5001")

```

```

print(" Health check: http://localhost:5001/health")
print(" Press Ctrl+C to stop\n")

try:
    # Import and run the Flask app
    from app import app, initialize_app

    if initialize_app():
        app.run(
            debug=os.getenv('DEBUG', 'True').lower() == 'true',
            host=os.getenv('HOST', '0.0.0.0'),
            port=int(os.getenv('PORT', 5001))
        )
    else:
        print("Failed to initialize application")

except KeyboardInterrupt:
    print("\n Server stopped by user")
except Exception as e:
    print(f"\n Server error: {str(e)}")

if __name__ == '__main__':
    main()

```

ARIMA and SARIMA Time Series Forecasting Module

Provides statistical forecasting methods for inventory demand prediction

```
import numpy as np
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.stattools import adfuller
from itertools import product
import warnings
warnings.filterwarnings('ignore')
```

```
class ARIMAPredictor:
```

```
    """ARIMA-based demand forecasting"""
```

```
    def __init__(self):
```

```
        self.model = None
```

```
        self.fitted_model = None
```

```
        self.order = None
```

```
    def check_stationarity(self, data):
```

```
        """Check if time series is stationary using ADF test"""
```

```
        try:
```

```
            result = adfuller(data)
```

```
            return {
```

```
                'is_stationary': result[1] < 0.05,
```

```
                'adf_statistic': result[0],
```

```
                'p_value': result[1],
```

```
                'critical_values': result[4]
```

```
            }
```

```
        except Exception as e:
```

```

    return {'is_stationary': False, 'error': str(e)}

def auto_find_order(self, data, seasonal=False, fast_mode=False):
    """Automatically find best ARIMA order using grid search"""
    try:
        # Convert to pandas Series
        if not isinstance(data, pd.Series):
            data = pd.Series(data)

        # Use smaller ranges for fast mode (long-term predictions)
        if fast_mode:
            p_range = range(0, 2)
            d_range = range(0, 2)
            q_range = range(0, 2)
        else:
            p_range = range(0, 3)
            d_range = range(0, 2)
            q_range = range(0, 3)

        best_aic = np.inf
        best_order = (1, 1, 1)
        best_seasonal_order = None

        # Grid search for non-seasonal
        for p, d, q in product(p_range, d_range, q_range):
            try:
                if seasonal:
                    # For SARIMA - use smaller ranges in fast mode
                    if fast_mode:
                        P_range = range(0, 2)
                        D_range = range(0, 1)
                        Q_range = range(0, 2)

```

```

else:
    P_range = range(0, 2)
    D_range = range(0, 2)
    Q_range = range(0, 2)
    m = 7 # Weekly seasonality

    for P, D, Q in product(P_range, D_range, Q_range):
        try:
            model = SARIMAX(data, order=(p, d, q),
                            seasonal_order=(P, D, Q, m),
                            enforce_stationarity=False,
                            enforce_invertibility=False)
            fitted = model.fit(dispatch=False, maxiter=50 if fast_mode else 100)
            if fitted.aic < best_aic:
                best_aic = fitted.aic
                best_order = (p, d, q)
                best_seasonal_order = (P, D, Q, m)
        except:
            continue

    else:
        # For ARIMA
        model = ARIMA(data, order=(p, d, q))
        fitted = model.fit(maxiter=50 if fast_mode else 100)
        if fitted.aic < best_aic:
            best_aic = fitted.aic
            best_order = (p, d, q)

    except:
        continue

    return best_order, best_seasonal_order
except Exception as e:
    # Default orders if grid search fails

```

```
return (1, 1, 1), (1, 1, 1, 7) if seasonal else None
```

```
def fit(self, data, order=None):
    """Fit ARIMA model to historical data"""
    try:
        # Convert to pandas Series
        if not isinstance(data, pd.Series):
            data = pd.Series(data)

        # Auto-find order if not provided
        if order is None:
            order, _ = self.auto_find_order(data, seasonal=False)

        self.order = order

        # Fit ARIMA model
        self.model = ARIMA(data, order=order)
        self.fitted_model = self.model.fit()

        return True
    except Exception as e:
        raise Exception(f"ARIMA fitting error: {str(e)}")

def predict(self, steps=1):
    """Make predictions for future steps"""
    try:
        if self.fitted_model is None:
            raise Exception("Model not fitted. Call fit() first.")

        forecast = self.fitted_model.forecast(steps=steps)

        # Ensure non-negative predictions
```

```

        predictions = [max(0, float(pred)) for pred in forecast]

    return predictions
except Exception as e:
    raise Exception(f"ARIMA prediction error: {str(e)}")

def predict_with_intervals(self, steps=1, alpha=0.05):
    """Make predictions with confidence intervals"""
    try:
        if self.fitted_model is None:
            raise Exception("Model not fitted. Call fit() first.")

        forecast_result = self.fitted_model.get_forecast(steps=steps)
        forecast = forecast_result.predicted_mean
        conf_int = forecast_result.conf_int(alpha=alpha)

        predictions = []
        for i in range(steps):
            predictions.append({
                'prediction': max(0, float(forecast.iloc[i])),
                'lower_bound': max(0, float(conf_int.iloc[i, 0])),
                'upper_bound': max(0, float(conf_int.iloc[i, 1])),
                'confidence': int((1 - alpha) * 100)
            })

        return predictions
    except Exception as e:
        raise Exception(f"ARIMA prediction with intervals error: {str(e)}")

def get_model_summary(self):
    """Get model summary statistics"""
    if self.fitted_model is None:

```

```

        return None

    return {
        'order': self.order,
        'aic': float(self.fitted_model.aic),
        'bic': float(self.fitted_model.bic),
        'model_type': 'ARIMA'
    }

class SARIMAPredictor:
    """SARIMA-based demand forecasting with seasonality"""

    def __init__(self, seasonal_period=7):
        self.model = None
        self.fitted_model = None
        self.order = None
        self.seasonal_order = None
        self.seasonal_period = seasonal_period

    def fit(self, data, order=None, seasonal_order=None):
        """Fit SARIMA model to historical data"""
        try:
            # Convert to pandas Series
            if not isinstance(data, pd.Series):
                data = pd.Series(data)

            # Auto-find orders if not provided
            if order is None or seasonal_order is None:
                predictor = ARIMAPredictor()
                order, seasonal_order = predictor.auto_find_order(data, seasonal=True)

            self.order = order

```

```

self.seasonal_order = seasonal_order

# Fit SARIMA model
self.model = SARIMAX(
    data,
    order=order,
    seasonal_order=seasonal_order,
    enforce_stationarity=False,
    enforce_invertibility=False
)
self.fitted_model = self.model.fit(dispatch=False)

return True
except Exception as e:
    raise Exception(f"SARIMA fitting error: {str(e)}")

def predict(self, steps=1):
    """Make predictions for future steps"""
    try:
        if self.fitted_model is None:
            raise Exception("Model not fitted. Call fit() first.")

        forecast = self.fitted_model.forecast(steps=steps)

        # Ensure non-negative predictions
        predictions = [max(0, float(pred)) for pred in forecast]

        return predictions
    except Exception as e:
        raise Exception(f"SARIMA prediction error: {str(e)}")

def predict_with_intervals(self, steps=1, alpha=0.05):

```

```

"""Make predictions with confidence intervals"""
try:
    if self.fitted_model is None:
        raise Exception("Model not fitted. Call fit() first.")

    forecast_result = self.fitted_model.get_forecast(steps=steps)
    forecast = forecast_result.predicted_mean
    conf_int = forecast_result.conf_int(alpha=alpha)

    predictions = []
    for i in range(steps):
        predictions.append({
            'prediction': max(0, float(forecast.iloc[i])),
            'lower_bound': max(0, float(conf_int.iloc[i, 0])),
            'upper_bound': max(0, float(conf_int.iloc[i, 1])),
            'confidence': int((1 - alpha) * 100)
        })

    return predictions
except Exception as e:
    raise Exception(f"SARIMA prediction with intervals error: {str(e)}")

def get_model_summary(self):
    """Get model summary statistics"""
    if self.fitted_model is None:
        return None

    return {
        'order': self.order,
        'seasonal_order': self.seasonal_order,
        'aic': float(self.fitted_model.aic),
        'bic': float(self.fitted_model.bic),

```

```
        'model_type': 'SARIMA'
    }
```

```
class HoltWintersPredictor:
```

```
    """Holt-Winters Exponential Smoothing predictor"""
```

```
    def __init__(self, seasonal_period=7):
```

```
        self.model = None
```

```
        self.fitted_model = None
```

```
        self.seasonal_period = seasonal_period
```

```
    def fit(self, data, seasonal='add', trend='add'):
```

```
        """Fit Holt-Winters model to historical data"""
```

```
        try:
```

```
            # Convert to pandas Series
```

```
            if not isinstance(data, pd.Series):
```

```
                data = pd.Series(data)
```

```
            # Need at least 2 seasonal periods
```

```
            if len(data) < self.seasonal_period * 2:
```

```
                # Use simple exponential smoothing if not enough data
```

```
                self.model = ExponentialSmoothing(
```

```
                    data,
```

```
                    trend=trend,
```

```
                    seasonal=None
```

```
                )
```

```
            else:
```

```
                # Use full Holt-Winters with seasonality
```

```
                self.model = ExponentialSmoothing(
```

```
                    data,
```

```
                    trend=trend,
```

```
                    seasonal=seasonal,
```

```

        seasonal_periods=self.seasonal_period
    )

    self.fitted_model = self.model.fit()
    return True

except Exception as e:
    raise Exception(f'Holt-Winters fitting error: {str(e)}')

def predict(self, steps=1):
    """Make predictions for future steps"""
    try:
        if self.fitted_model is None:
            raise Exception("Model not fitted. Call fit() first.")

        forecast = self.fitted_model.forecast(steps=steps)

        # Ensure non-negative predictions
        predictions = [max(0, float(pred)) for pred in forecast]

        return predictions
    except Exception as e:
        raise Exception(f'Holt-Winters prediction error: {str(e)}')

def predict_with_intervals(self, steps=1, alpha=0.05):
    """Make predictions with confidence intervals"""
    try:
        if self.fitted_model is None:
            raise Exception("Model not fitted. Call fit() first.")

        forecast = self.fitted_model.forecast(steps=steps)

```

```

# Calculate prediction intervals using standard error
# Simple approximation:  $\pm 1.96 * \text{std}$  for 95% CI
residuals = self.fitted_model.resid
std_error = np.std(residuals)
z_score = 1.96 # 95% confidence

predictions = []
for i in range(steps):
    pred_value = float(forecast.iloc[i]) if hasattr(forecast, 'iloc') else float(forecast[i])
    margin = z_score * std_error * np.sqrt(i + 1) # Increases with forecast horizon

    predictions.append({
        'prediction': max(0, pred_value),
        'lower_bound': max(0, pred_value - margin),
        'upper_bound': max(0, pred_value + margin),
        'confidence': int((1 - alpha) * 100)
    })

return predictions
except Exception as e:
    raise Exception(f'Holt-Winters prediction with intervals error: {str(e)}")

def get_model_summary(self):
    """Get model summary statistics"""
    if self.fitted_model is None:
        return None

    return {
        'seasonal_period': self.seasonal_period,
        'aic': float(self.fitted_model.aic) if hasattr(self.fitted_model, 'aic') else None,
        'model_type': 'Holt-Winters'
    }

```

```

class EnsemblePredictor:
    """Ensemble predictor combining ARIMA, SARIMA, and Holt-Winters"""

    def __init__(self, lstm_predictor=None):
        self.arima = ARIMAPredictor()
        self.sarima = SARIMAPredictor()
        self.holt_winters = HoltWintersPredictor()
        self.lstm_predictor = lstm_predictor
        # Equal weights for all three models
        self.weights = {'arima': 0.33, 'sarima': 0.33, 'holt_winters': 0.34}

    def fit(self, data):
        """Fit all models"""
        results = {}

        # Fit ARIMA
        try:
            self.arima.fit(data)
            results['arima'] = 'success'
        except Exception as e:
            results['arima'] = f'failed: {str(e)}'

        # Fit SARIMA
        try:
            self.sarima.fit(data)
            results['sarima'] = 'success'
        except Exception as e:
            results['sarima'] = f'failed: {str(e)}'

        return results

```

```

def predict(self, data, steps=1):
    """Make ensemble predictions using ARIMA, SARIMA, and Holt-Winters"""
    predictions = []
    weights_used = []

    # ARIMA prediction
    try:
        self.arima.fit(data)
        arima_pred = self.arima.predict(steps)
        predictions.append(('arima', arima_pred, self.weights['arima']))
        weights_used.append(self.weights['arima'])
    except:
        pass

    # SARIMA prediction
    try:
        self.sarima.fit(data)
        sarima_pred = self.sarima.predict(steps)
        predictions.append(('sarima', sarima_pred, self.weights['sarima']))
        weights_used.append(self.weights['sarima'])
    except:
        pass

    # Holt-Winters prediction
    try:
        self.holt_winters.fit(data)
        hw_pred = self.holt_winters.predict(steps)
        predictions.append(('holt_winters', hw_pred, self.weights['holt_winters']))
        weights_used.append(self.weights['holt_winters'])
    except:
        pass

```

```

# Combine predictions
if not predictions:
    raise Exception("All models failed to predict")

# Normalize weights
total_weight = sum(weights_used)

ensemble_predictions = []
for step in range(steps):
    weighted_sum = 0
    for model_name, preds, weight in predictions:
        weighted_sum += preds[step] * (weight / total_weight)
    ensemble_predictions.append(max(0, weighted_sum))

return {
    'ensemble_predictions': ensemble_predictions,
    'individual_predictions': {
        name: preds for name, preds, _ in predictions
    },
    'models_used': [name for name, _, _ in predictions]
}

def compare_models(self, data, test_size=7):
    """Compare performance of ARIMA, SARIMA, and Holt-Winters models"""
    if len(data) < test_size + 14:
        raise Exception("Insufficient data for comparison")

    train_data = data[:-test_size]
    test_data = data[-test_size:]

    results = {}

```

```
# Test ARIMA
```

```
try:
```

```
    self.arima.fit(train_data)
```

```
    arima_pred = self.arima.predict(test_size)
```

```
    arima_mae = np.mean(np.abs(np.array(arima_pred) - np.array(test_data)))
```

```
    results['arima'] = {'predictions': arima_pred, 'mae': float(arima_mae)}
```

```
except Exception as e:
```

```
    results['arima'] = {'error': str(e)}
```

```
# Test SARIMA
```

```
try:
```

```
    self.sarima.fit(train_data)
```

```
    sarima_pred = self.sarima.predict(test_size)
```

```
    sarima_mae = np.mean(np.abs(np.array(sarima_pred) - np.array(test_data)))
```

```
    results['sarima'] = {'predictions': sarima_pred, 'mae': float(sarima_mae)}
```

```
except Exception as e:
```

```
    results['sarima'] = {'error': str(e)}
```

```
# Test Holt-Winters
```

```
try:
```

```
    self.holt_winters.fit(train_data)
```

```
    hw_pred = self.holt_winters.predict(test_size)
```

```
    hw_mae = np.mean(np.abs(np.array(hw_pred) - np.array(test_data)))
```

```
    results['holt_winters'] = {'predictions': hw_pred, 'mae': float(hw_mae)}
```

```
except Exception as e:
```

```
    results['holt_winters'] = {'error': str(e)}
```

```
results['actual'] = list(test_data)
```

```
return results
```

Model Loader

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from flask_jwt_extended import JWTManager, jwt_required, get_jwt_identity
import os
import logging
from datetime import timedelta
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# Import our modules
# from models.database import db
# from routes.auth import auth_bp
from config import Config
from model_loader import ModelLoader
from predictor import InventoryPredictor

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

app = Flask(__name__)

# Configuration
app.config['JWT_SECRET_KEY'] = os.getenv('JWT_SECRET_KEY')
app.config['JWT_ACCESS_TOKEN_EXPIRES'] =
timedelta(seconds=int(os.getenv('JWT_ACCESS_TOKEN_EXPIRES', 3600)))

# Initialize extensions
jwt = JWTManager(app)
```

```
CORS(app, origins=os.getenv('CORS_ORIGINS', 'http://localhost:3000').split(','))
```

```
# Global variables for ML model
```

```
model_loader = ModelLoader()
```

```
predictor = None
```

```
def initialize_app():
```

```
    """Initialize the application with model and predictor"""
```

```
    global predictor
```

```
    try:
```

```
        # Skip database connection for now
```

```
        logger.info(" Running without database connection")
```

```
        # Load ML model and scaler
```

```
        model_loaded = model_loader.load_model()
```

```
        scaler_loaded = model_loader.load_scaler()
```

```
        if model_loaded and scaler_loaded:
```

```
            predictor = InventoryPredictor(
```

```
                model_loader.get_model(),
```

```
                model_loader.get_scaler(),
```

```
                model_loader.device
```

```
            )
```

```
            logger.info("ML model initialized successfully")
```

```
            return True
```

```
        else:
```

```
            logger.warning("ML model not loaded, continuing without predictions")
```

```
            return True
```

```
    except Exception as e:
```

```
        logger.error(f' Failed to initialize application: {str(e)}')
```

```

        return False

# Register blueprints
# app.register_blueprint(auth_bp, url_prefix='/api/auth')

# Health check endpoint
@app.route('/health', methods=['GET'])
def health_check():
    """Health check endpoint"""
    try:
        return jsonify({
            'status': 'healthy',
            'database_connected': False, # Running without database
            'model_loaded': model_loader.model is not None,
            'scaler_loaded': model_loader.scaler is not None,
            'predictor_ready': predictor is not None
        })
    except Exception as e:
        return jsonify({
            'status': 'error',
            'message': str(e)
        }), 500

# Model info endpoint
@app.route('/model-info', methods=['GET'])
def model_info():
    """Get information about the loaded model"""
    return jsonify({
        'model_type': 'LSTM',
        'input_size': Config.INPUT_SIZE,
        'hidden_size': Config.HIDDEN_SIZE,
        'num_layers': Config.NUM_LAYERS,

```

```

        'sequence_length': Config.SEQUENCE_LENGTH,
        'framework': 'PyTorch',
        'device': str(model_loader.device),
        'model_loaded': model_loader.model is not None,
        'scaler_loaded': model_loader.scaler is not None,
        'predictor_ready': predictor is not None
    })

# Prediction endpoints (no authentication for now)
@app.route('/predict-single', methods=['POST'])
def predict_single():
    """Make a single prediction"""
    try:
        if predictor is None:
            return jsonify({'error': 'Predictor not initialized'}), 500

        data = request.get_json()
        historical_data = data.get('historical_data', [])

        if not historical_data:
            return jsonify({'error': 'Historical data is required'}), 400

        # Make single prediction
        prediction = predictor.predict_single(historical_data)

        return jsonify({
            'prediction': prediction,
            'model_type': 'LSTM',
            'status': 'success'
        })

    except Exception as e:

```

```

        return jsonify({'error': str(e)}), 500

@app.route('/predict', methods=['POST'])
def predict_demand():
    """Predict inventory demand"""
    try:
        if predictor is None:
            return jsonify({'error': 'Predictor not initialized'}), 500

        data = request.get_json()

        # Extract parameters
        historical_data = data.get('historical_data', [])
        prediction_days = data.get('prediction_days',
Config.DEFAULT_PREDICTION_DAYS)

        if not historical_data:
            return jsonify({'error': 'Historical data is required'}), 400

        if len(historical_data) < Config.MIN_HISTORICAL_DATA_POINTS:
            return jsonify({
                'error': f'Insufficient historical data (minimum
{Config.MIN_HISTORICAL_DATA_POINTS} points required)'
            }), 400

        # Make predictions
        predictions = predictor.predict_multiple(historical_data, prediction_days)

        return jsonify({
            'predictions': predictions,
            'prediction_days': prediction_days,
            'model_type': 'LSTM',

```

```

        'status': 'success'
    })

except Exception as e:
    return jsonify({'error': str(e)}), 500

@app.route('/predict-weekly', methods=['POST'])
def predict_weekly_demand():
    """Predict weekly inventory demand"""
    try:
        if predictor is None:
            return jsonify({'error': 'Predictor not initialized'}), 500

        data = request.get_json()

        # Extract parameters
        historical_data = data.get('historical_data', [])
        weeks_ahead = data.get('weeks_ahead', Config.DEFAULT_WEEKS_AHEAD)

        if not historical_data:
            return jsonify({'error': 'Historical data is required'}), 400

        # Make weekly predictions
        weekly_predictions = predictor.predict_weekly(historical_data, weeks_ahead)

        return jsonify({
            'weekly_predictions': weekly_predictions,
            'weeks_ahead': weeks_ahead,
            'model_type': 'LSTM',
            'status': 'success'
        })

```

```

except Exception as e:
    return jsonify({'error': str(e)}), 500

@app.route('/analyze-trends', methods=['POST'])
def analyze_trends():
    """Analyze trends in historical data"""
    try:
        if predictor is None:
            return jsonify({'error': 'Predictor not initialized'}), 500

        data = request.get_json()
        historical_data = data.get('historical_data', [])

        if not historical_data or len(historical_data) < 7:
            return jsonify({'error': 'Insufficient historical data (minimum 7 points required)'}), 400

        # Analyze trends
        trend_analysis = predictor.analyze_trends(historical_data)
        trend_analysis['status'] = 'success'

        return jsonify(trend_analysis)

    except Exception as e:
        return jsonify({'error': str(e)}), 500

# Error handlers
@app.errorhandler(404)
def not_found(error):
    return jsonify({'error': 'Endpoint not found'}), 404

@app.errorhandler(500)
def internal_error(error):

```

```

    return jsonify({'error': 'Internal server error'}), 500

@jwt.expired_token_loader
def expired_token_callback(jwt_header, jwt_payload):
    return jsonify({'message': 'Token has expired'}), 401

@jwt.invalid_token_loader
def invalid_token_callback(error):
    return jsonify({'message': 'Invalid token'}), 401

@jwt.unauthorized_loader
def missing_token_callback(error):
    return jsonify({'message': 'Authorization token is required'}), 401

if __name__ == '__main__':
    # Initialize application
    if initialize_app():
        logger.info("Starting Flask API server...")
        app.run(
            debug=os.getenv('DEBUG', 'True').lower() == 'true',
            host=os.getenv('HOST', '0.0.0.0'),
            port=int(os.getenv('PORT', 5001))
        )
    else:
        logger.error("Failed to initialize application. Please check configuration.")

```

```

import torch
import torch.nn as nn
import pickle
import os
from sklearn.preprocessing import MinMaxScaler
from config import Config

class LSTMModel(nn.Module):
    """LSTM Model for inventory demand prediction"""
    def __init__(self, input_size=1, hidden_size=64, num_layers=2, output_size=1):
        super(LSTMModel, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers

        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        # Initialize hidden state with zeros
        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(x.device)
        c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(x.device)

        # Forward propagate LSTM
        out, _ = self.lstm(x, (h0, c0))

        # Apply dropout and get the last output
        out = self.dropout(out[:, -1, :])
        out = self.fc(out)
        return out

```

```

class ModelLoader:
    def __init__(self):
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model = None
        self.scaler = None

    def load_model(self, model_path=None):
        """Load the trained LSTM model"""
        if model_path is None:
            model_path = Config.MODEL_PATH

        try:
            # Initialize model
            self.model = LSTMMModel(
                input_size=Config.INPUT_SIZE,
                hidden_size=Config.HIDDEN_SIZE,
                num_layers=Config.NUM_LAYERS,
                output_size=Config.OUTPUT_SIZE
            ).to(self.device)

            # Load weights
            if os.path.exists(model_path):
                state_dict = torch.load(model_path, map_location=self.device)
                self.model.load_state_dict(state_dict)
                self.model.eval()
                print(f"Model loaded from {model_path}")
                return True
            else:
                print(f"Model file not found: {model_path}")
                return False

```

```

except Exception as e:
    print(f"Error loading model: {str(e)}")
    return False

def load_scaler(self, scaler_path=None):
    """Load the data scaler"""
    if scaler_path is None:
        scaler_path = Config.SCALER_PATH

    try:
        if os.path.exists(scaler_path):
            with open(scaler_path, 'rb') as f:
                self.scaler = pickle.load(f)
            print(f"Scaler loaded from {scaler_path}")
            return True
        else:
            # Create a default scaler
            self.scaler = MinMaxScaler()
            print(f"Scaler file not found: {scaler_path}. Using default MinMaxScaler.")
            return True

    except Exception as e:
        print(f"Error loading scaler: {str(e)}")
        self.scaler = MinMaxScaler()
        return True

def save_scaler(self, scaler_path=None):
    """Save the fitted scaler"""
    if scaler_path is None:
        scaler_path = Config.SCALER_PATH

    try:

```

```

        os.makedirs(os.path.dirname(scaler_path), exist_ok=True)
        with open(scaler_path, 'wb') as f:
            pickle.dump(self.scaler, f)
        print(f"Scaler saved to {scaler_path}")
        return True
    except Exception as e:
        print(f"Error saving scaler: {str(e)}")
        return False

    def get_model(self):
        """Get the loaded model"""
        return self.model

    def get_scaler(self):
        """Get the loaded scaler"""
        return self.scaler

    def is_ready(self):
        """Check if model and scaler are loaded"""
        return self.model is not None and self.scaler is not None

```